Increment, switch এবং পুনরাবৃত্তির লুপ

পাঠ 12 — 1IOT, SST-ICT-01 — যৌগিক মান নির্ধারণ, x++ ও ++x, switch গঠন, for, while ও do while লুপ

19/10/2026

1 আজকের বিষয়

দুটি বিষয়: ভেরিয়েবলের **increment** (**incremento**, বাড়ানো) আর **পুনরাবৃত্তির লুপ** (**cicli di ripetizione**)। মাঝে, `if`-এর সাথে দেখা **সিদ্ধান্তের গঠনগুলো** সম্পূর্ণ করতে, **switch** গঠনও আছে। পাঠের শেষে **onlinegdb**-তে সরাসরি পরীক্ষা করা কিছু উদাহরণ।

2 Increment

একটি ভেরিয়েবলকে **increment করা** মানে তাতে **একটি পরিমাণ যোগ করা**। একটি উদাহরণ দিয়ে শুরু করি:

```
int x;
x = 6;
x = x + 3;
```

শেষে x-এর মান কত? **9**। কারণ হলো **সমান চিহ্নের** কাজ, যা অনেকবার বলা হয়েছে: সমান চিহ্ন ডানের **মান নিয়ে** বাঁয়ের ভেরিয়েবলে রাখে। ডানে আছে `x + 3`: x হলো 6, তাই $6 + 3 = 9$, আর 9 x-এ রাখা হয়। ভেরিয়েবলটি নতুন মান দিয়ে **প্রতিস্থাপিত** হয়।

সংজ্ঞা

`x = x + a` কাজটিকে, যেখানে `a` যেকোনো সংখ্যা (বা আরেকটি ভেরিয়েবল), বলে **increment**: ভেরিয়েবল নিয়ে তাতে **একটি মান যোগ করা** হয়।

3 যৌগিক মান নির্ধারণ: +=, -=, *=, /=

প্রোগ্রামাররা **অলস**, এবং তার ভালো কারণ আছে: **যত কম লেখা**, **ভুলের সম্ভাবনা** তত কম। এর মানে এই নয় যে কম লিখলে কখনো ভুল হয় না, তবে ছোট লাইনের চেয়ে লম্বা লাইনে ভুলের সুযোগ বেশি। তাই C-তে increment-এর একটি **আরও সংক্ষিপ্ত রূপ** আছে:

```
x = x + a;
// সমানভাবে এভাবে লেখা যায়:
x += a;
```

`+=` প্রতীকের মানে: **x নাও**, **a যোগ করো** এবং **ফলাফল** আবার x-এ **রাখো**। যেমন `x = 7;` আর তারপর `x += 3;` দিলে x হয় **10**। সাবধান: increment বলা হয় যখন **একই ভেরিয়েবলে** মান যোগ হয়; **একাধিক**

আলাদা মান যোগ করতে হলে সেটি আর increment নয়, একটি সাধারণ যোগ।

> **টীকা**
>
> a শুধু যেকোনো একটি সংখ্যা বোঝানোর উপায়: 3, 4, 12, বা একটি ভেরিয়েবল হতে পারে। যেমন `int a; a = 12;`-এর পরে `x = x + 12;` বা `x += 12;` লিখলে একই ফলাফল।

একই রূপ অন্য কাজের জন্যও চলে:

সংক্ষিপ্ত রূপ	সমান	উদাহরণ	ফলাফল
`x += 3;`	`x = x + 3;`	x হলো 7	x হয় 10
`x -= 3;`	`x = x - 3;`	x হলো 20	x হয় 17
`x *= 2;`	`x = x * 2;`	x হলো 5	x হয় 10
`x /= 2;`	`x = x / 2;`	x হলো 10	x হয় 5

`-=` হলো decrement (**decremento**, কমানো), তবে এটি increment পরিবারেরই অংশ। সাধারণ নিয়ম: `x = x` **কাজ সংখ্যা** লিখতে হলে অপারেটরকে সমান চিহ্নের আগে এনে দ্বিতীয় `x` সরানো যায়।

> **টীকা**
>
> বিট ধরে কাজের অপারেটরের জন্যও একই রূপ আছে, যেমন `x = x & ...`-এর বদলে `x &= ...`: এটি ডেটা মাস্কিংয়ের জন্য, যা firmware করার সময় একক বিট নিয়ে কাজে দেখব।

4 একক increment: x++ এবং x–

শুধু যোগ 1 আর বিয়োগ 1-এর জন্য আরও ছোট একটি রূপ আছে:

লম্বা রূপ	সংক্ষিপ্ত রূপ	একক রূপ
`x = x + 1;`	`x += 1;`	`x++;`
`x = x - 1;`	`x -= 1;`	`x--;`

পরপর দুটি যোগ চিহ্ন ভেরিয়েবলকে 1 বাড়ায়, দুটি বিয়োগ চিহ্ন 1 কমায়। যেমন `x = 7; x++;` x-কে 8 করে। আমরা লুপে এটি অনেক ব্যবহার করব।

> **সতর্কতা**
>
> `++` চলে শুধু যোগ 1-এর জন্য। 2 যোগ করতে `x+++` নেই: লেখা হয় `x += 2` বা `x = x + 2`।

5 x++ না ++x: কথাটি হলো কখন

++ ভেরিয়েবলের **পরে** (x++) বা **আগে** (++x) লেখা যায়। একা, এক লাইনে, দুটোই একই কাজ করে: **1 বাড়ায়**। পার্থক্য দেখা যায় যখন মানটি **ব্যবহারও** হয়, যেমন একটি মান নির্ধারণে। ক্লাসে আমরা এই দুটি কোড তুলনা করেছি:

```
int x;
x = 7;
int y;
y = x++;
```

```
int x;
x = 7;
int y;
y = ++x;
```

সংজ্ঞা

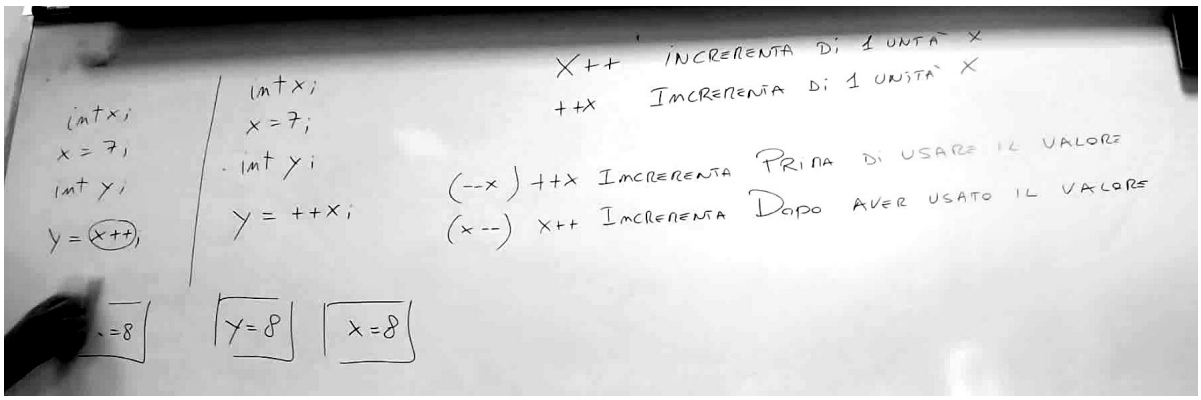
- ++x (ও --x): মান ব্যবহারের **আগে** **বাড়ায়**।
- x++ (ও x--): মান ব্যবহারের **পরে** **বাড়ায়**।

কোড	কী ঘটে	ফলাফল
y = x++;	আগে 7 y-তে কপি, তারপর x বাড়ে	y = 7, x = 8
y = ++x;	আগে x বেড়ে 8, তারপর y-তে কপি	y = 8, x = 8

দুই ক্ষেত্রেই **শেষে** x **হয়** **8**: শুধু y-তে **যে** **মান** **যায়** তা বদলায়।

সতর্কতা

ক্লাসে প্রথম ব্যাখ্যায় দুটি ক্ষেত্র **উল্টে** **গিয়েছিল**; শিক্ষক বোর্ডে **সঙ্গে** **সঙ্গে** **ঠিক** **করেন** (ছবি 1)। তোমার নোটে যদি লেখা থাকে y = x++;-এ y-ও 8, তবে **ঠিক** **করে** **নাও**: x++-এ y পায় **পুরোনো** মান, অর্থাৎ **7**।



ছবি/নকশা 1: দুটি কোড ও সঠিক নিয়মসহ বোর্ড: ++x মান ব্যবহারের আগে বাড়ায়, x++ ব্যবহারের পরে বাড়ায়। (প্রথম কোডের নিচের ঘরটি আংশিক হাতে ঢাকা।)

টীকা

একটি কৌতূহল: polling ও interrupt। ক্লাসের লেখা শেষ হওয়ার অপেক্ষায় শিক্ষক মাঝে মাঝে জিজ্ঞেস করেছেন "এগোতে পারি?"। কম্পিউটারে এই কৌশলকে বলে **polling**: একটি যন্ত্রকে **বারবার জিজ্ঞেস করা** তার কাজ শেষ কিনা। বিকল্প হলো **interrupt** (ইতালীয়তে **interruzione**): যন্ত্র নিজেই শেষ হলে **জানিয়ে দেয়**।

6 switch গঠন

6.1 সমস্যা: অনেক বেশি if

`if` একটি **শর্ত** অনুযায়ী কোন কোড চলবে তা বাছে, যা শুধু **সত্য বা মিথ্যা** হতে পারে। কোনো প্রোগ্রাম 1 থেকে 10 একটি সংখ্যা চেয়ে সংখ্যা অনুযায়ী 10টি আলাদা কাজ করলে, `if` দিয়ে একটি লম্বা শৃঙ্খল লিখতে হয়: `if (dato == 1) ... else if (dato == 2) ... else if (dato == 3) ...` এভাবে চলতে থাকে। এটি কাজ করে, কিন্তু **অসুবিধাজনক** ও **গোলমেলে**, এবং **ভুল** সহজ করে।

সংজ্ঞা

switch এমন একটি গঠন, যা একটি **ভেরিয়েবলের মান** অনুযায়ী **কী করতে হবে** তা **গুছিয়ে** বাছে: এটি অনেক `else`-সহ একটি `if`-এর মতো, কিন্তু অনেক বেশি স্পষ্ট ও সামলানো সহজ বিন্যাসে।

6.2 গঠন

```
switch (variabile_da_valutare)
{
    case opzione1:
        istruzione1;
        istruzione2;
        istruzione3;
        break;
    case opzione2:
        istruzione1;
        break;
    default:
        istruzione1;
        istruzione2;
}
```

- `switch`-এর **গোল বন্ধনীতে** থাকে **যাচাই করার ভেরিয়েবল** (**variabile da valutare**)।
- প্রতিটি `case` হলো ভেরিয়েবলের একটি **সম্ভাব্য মান**, তারপর **কোলন**। `switch` ও `case` গঠনের দুটি **মূল শব্দ**।
- "istruzione" C-র **কোনো নির্দেশ নয়**: এটি **যেকোনো নির্দেশের ধারা** বোঝায়, এক, দুই, পঞ্চাশ।
- প্রতিটি `case` শেষ হয় `break;` দিয়ে।
- `default` শেষ সম্ভাবনা: ভেরিয়েবল কোনো `case`-এর সাথে **না মিললে** চলে।

- case যত খুশি থাকতে পারে: 4, 7, 18, 25। বোর্ডে জায়গার জন্য দুটি ছিল।

6.3 উদাহরণ: তিনটি পছন্দের একটি মেনু

```
int scelta;
printf("Inserisci una scelta");
scanf("%d", &scelta);

switch (scelta)
{
    case 1:
        printf("Hai scelto la prima opzione");
        break;
    case 2:
        printf("Hai scelto la seconda opzione");
        break;
    case 3:
        printf("Hai fatto la terza scelta");
        break;
    default:
        printf("Non hai fatto una scelta valida");
}
```

প্রোগ্রাম scanf দিয়ে একটি সংখ্যা পড়ে। switch scelta (পছন্দ) ভেরিয়েবলকে ক্ষেত্রগুলোর সাথে একে একে তুলনা করে: 1, 2, 3-এর সাথে। সমতা পেলে সেই ক্ষেত্রের কোড চালায়। ব্যবহারকারী 1, 2 বা 3 ছাড়া অন্য মান দিলে default চলে: "Non hai fatto una scelta valida" (তুমি বৈধ পছন্দ করোনি)।

টীকা

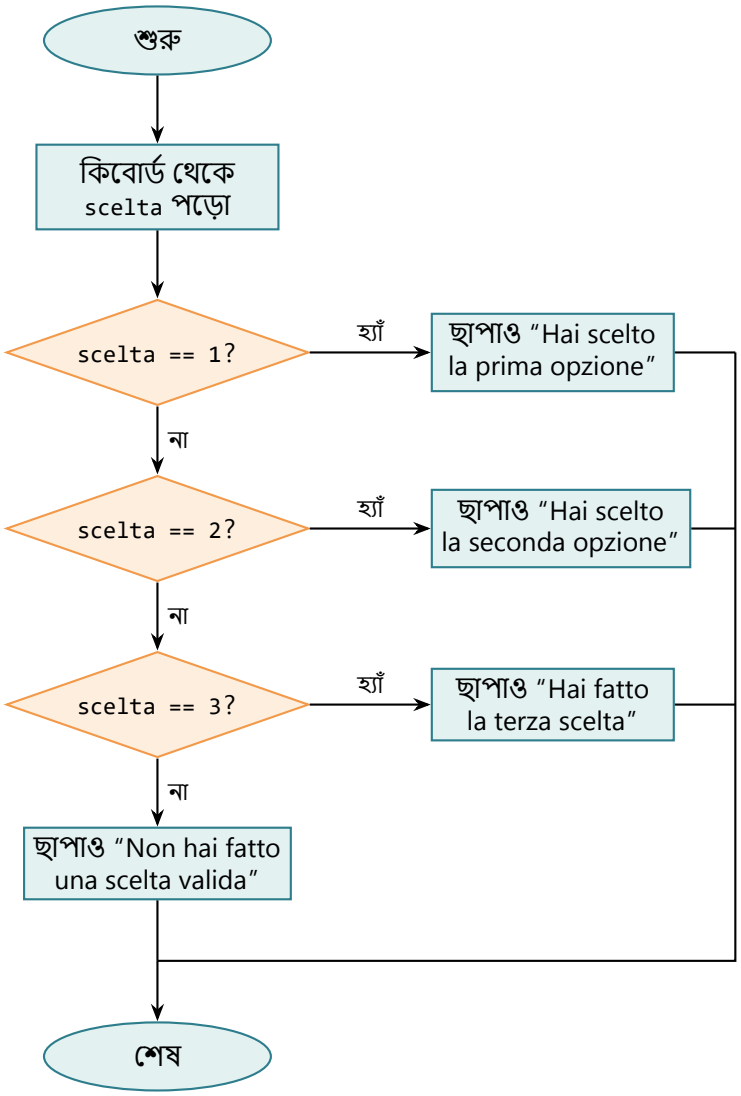
printf-এর ভেতরের লেখা প্রোগ্রামার ঠিক করে: তৃতীয়টি ইচ্ছে করে অন্যভাবে লেখা, যাতে দেখা যায় সবসময় একই হতে হবে না। জায়গার জন্য প্রতিটি case-এ একটি নির্দেশ, তবে একাধিকও থাকতে পারে।

6.4 break: গঠন থামায়

সংজ্ঞা

break গঠনের চলা থামায়: যন্ত্র এটি পেলে switch থেকে বেরিয়ে বন্ধ বন্ধনীর পরের কোডে এগোয়।

scelta 1 হলে যন্ত্র printf("Hai scelto la prima opzione"); চালায়, তারপর break পেয়ে বের হয়। কিন্তু break ছাড়া কী হয়? প্রথম printf-এর পরে যন্ত্র নেমে যায়: case 2: পেয়ে আবার তুলনা করে না, এগিয়ে গিয়ে ক্ষেত্র 2-এর printf-ও চালায়, এভাবে যতক্ষণ না break পায় বা শেষে পৌঁছায়।



ছবি/নকশা 2: উদাহরণের switch-এর ফ্লোচার্ট: ক্ষেত্রগুলো একে একে তুলনা হয়; break সঙ্গে সঙ্গে গঠনের শেষে নিয়ে যায়।

সতর্কতা

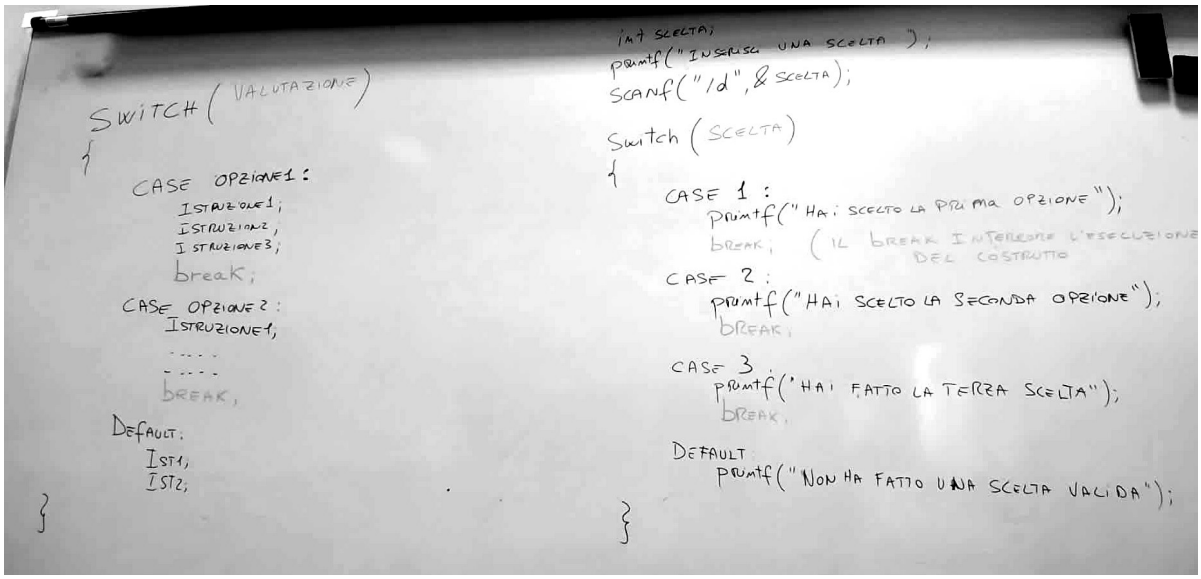
break **না দিলে** switch পাওয়া ক্ষেত্রের নিচের **সবকিছু চালায়**। খুব সাবধান থাকতে হবে: **break সবসময় দিতে হবে**, শুধু যখন ঠিক এই আচরণই চাই তখন ছাড়া (নিচে দেখো)।

উদাহরণ

একাধিক পছন্দ একসাথে করা। এই আচরণ একাধিক ক্ষেত্রে একই কাজ করতে কাজে লাগে। scelta 1 **অথবা** 2 হলে একই লেখা চাইলে দুটি case একটির নিচে একটি লিখি:

```
case 1:
case 2:
    printf("Hai scelto o la prima o la seconda");
    break;
```

scelta 1 হলে যন্ত্র printf পর্যন্ত "নামে"; 2 হলে সরাসরি পৌঁছায়। এভাবে switch **বহু পছন্দের** জন্য একটি খুব শক্তিশালী হাতিয়ার হয়।



ছবি/নকশা 3: বোর্ড: বাঁয়ে switch-এর সাধারণ গঠন, ডানে তিনটি পছন্দের মেনুর উদাহরণ এবং "break গঠনের চলা থামায়" টীকা।

টীকা

সবাই হ্যাঁ, অথবা অন্তত একজন না। পুরো ক্লাসের লেখা শেষ কিনা জানতে শিক্ষক লজিক দিয়ে একটি উদাহরণ দেন: "সবার শেষ" একটি **AND**: প্রথমজন হ্যাঁ বলে **এবং** দ্বিতীয়জন হ্যাঁ বলে **এবং**... শেষজনও হ্যাঁ বলে। এর উল্টো, "সবার শেষ হয়নি", একটি **OR**: প্রথমজন না বলে **অথবা** দ্বিতীয়জন না বলে **অথবা**... একটি বাক্য থেকে তার উল্টোতে গেলে **AND হয় OR** আর **প্রতিটি হ্যাঁ হয় না**: এটি **De Morgan-এর উপপাদ্য**। `do while`-এ আবার পাব।
শুধু "সবাই" লেখা যায় না? আরও বুদ্ধিমানভাবে লেখা যায়, কিন্তু যাচাই করতেই হবে: **কম লেখার** উপায় খুঁজতে হবে, কারণ যত বেশি লেখা তত বেশি ভুলের সম্ভাবনা। **যন্ত্র ভুল করে না**: ভুল করে **প্রোগ্রামার**।

টীকা

জিনিসের উপকারিতার সীমা টেনো না। শিক্ষকের দাদা ভাবতেন **রিমোট কন্ট্রোল** কার কাজে লাগে: আজ কেউ চ্যানেল বদলাতে উঠতে বললে পাগলামি মনে হবে। সবচেয়ে অকেজো মনে হওয়া জিনিসও **খুব কাজের** হতে পারে।

7 পুনরাবৃত্তির লুপ

কম্পিউটার একই কাজ বারবার করতে **বিরক্ত হয় না**, আর সেগুলো **সবসময় একইভাবে** করতে খুব দক্ষ। **পুনরাবৃত্তির লুপ** তাকে একটি কাজ 10, 100, 1000 বার করতে বলার জন্য। 100টি সংখ্যা নিতে হলে 100 বার `printf` ও `scanf` না লিখে এমন একটি গঠন লিখি যা একই কাজ 100 বার **পুনরাবৃত্তি করে**।

সংজ্ঞা

C-তে পুনরাবৃত্তির গঠন **তিনটি**: `for`, `while` ও `do while`। প্রথম দুটির গঠন আলাদা হলেও **একই কাজ করে**। তৃতীয়টি `while`-**এর একটি রূপ**।

7.1 for

```
for (condizione di partenza; condizione di ripetizione; modificatore)
{
    istruzioni da ripetere;
}
```

(**condizione di partenza** = শুরুর শর্ত, **condizione di ripetizione** = পুনরাবৃত্তির শর্ত, **modificatore** = পরিবর্তক, **istruzioni da ripetere** = পুনরাবৃত্তির নির্দেশ।) বন্ধনীর ভেতরে **তিনটি অংশ**, **সেমিকোলন** দিয়ে আলাদা:

1. **শুরুর শর্ত**: গোনা কোথা থেকে শুরু হয়;
2. **পুনরাবৃত্তির শর্ত**: কতক্ষণ চলবে;
3. **পরিবর্তক**: প্রতি ঘূর্ণনে গণনা কীভাবে বদলায়।

সতর্কতা

তিনটি অংশ **সেমিকোলন** দিয়ে আলাদা, কমা দিয়ে **নয়** এবং কোলন দিয়ে **নয়**: নাহলে কম্পাইলার **ভুল** দেখায়। এটি সহজ ভুল: বোর্ডেও (ছবি 5), 30 লাফের উদাহরণে, দ্বিতীয় সেমিকোলনের বদলে একটি কমা চলে গিয়েছিল।

উদাহরণ

স্বেচ্ছাসেবকের লাফ। ক্লাসে একজন জোরে গুনে 10টি লাফ দিয়েছে। লাফ ছাড়া সে কী করেছে? **হিসাব রেখেছে**: প্রথম লাফ, দ্বিতীয়, তৃতীয়...আর দশে পৌঁছে **থেমেছে**। আর সে নিজেই **এক থেকে শুরু করার** সিদ্ধান্ত নিয়েছে: কেউ বলেনি। 100 পর্যন্ত যেতে হলে সে যদি 90 বা 99 থেকে গোনা শুরু করত, "100-এ থামো" নিয়ম তবু মানা হতো, কিন্তু লাফ হতো খুব কম: তাই **শুরুর বিন্দু** ঠিক করতে হয়। এই হলো `for`-এর তিন অংশ:

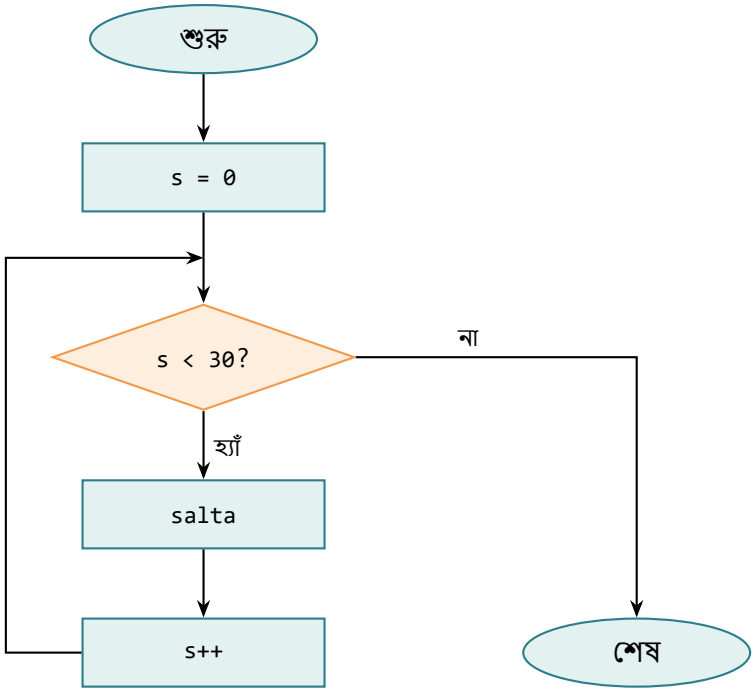
- গণনার একটি **শুরুর বিন্দু**, একটি **ভেরিয়েবলে** রাখা;
- **কতক্ষণ** পুনরাবৃত্তি: গণনা শেষ বিন্দুর চেয়ে ছোট থাকা পর্যন্ত লাফ চলে;
- প্রতি লাফে গণনা কত **বাড়বে**: এক।

C-তে লিখি, 30 লাফ আর `s` ভেরিয়েবল দিয়ে ("salti" = লাফ থেকে; **salta** = লাফ দাও):

```
for (int s = 0; s < 30; s++)
{
    salta;
}
```

- `int s = 0`: `s` ভেরিয়েবল **তৈরি করে** এবং **শুরুর মান** দেয়। শুরু হয় **শূন্য** থেকে: কম্পিউটার বিশেষজ্ঞরা **শূন্য থেকে গোনে**, কারণ শুরুর আগে কোনো লাফ হয়নি। ভেরিয়েবলটি `for`-এর আগেও তৈরি করা যায় (`int s;`) এবং শুধু `s = 0` লেখা যায়।
- `s < 30`: `s` 30-এর ছোট **থাকা পর্যন্ত পুনরাবৃত্তি**।
- `s++`: প্রতি ঘূর্ণনে `s` **1 বাড়ে**। `s = s + 1` বা `s += 1`-ও লেখা যেত; 2 করে লাফের জন্য `s += 2`।
- `salta;` বোঝায় পুনরাবৃত্তির **যেকোনো নির্দেশ**।

লাফ হয় `s`-এর মান 0, 1, 2, 3, … থেকে 29 পর্যন্ত: **30টি লাফ**। `s` 30 হলে `s < 30` শর্ত মিথ্যা এবং লুপ **শেষ হয়**।



ছবি/নকশা 4: 30 লাফের লুপের ফ্লোচার্ট। অংশ 7.3-এর `while`-এর জন্যও একই: শুধু লেখার ধরন বদলায়।

7.2 শুরু ও ধাপে সাবধান

উদাহরণ

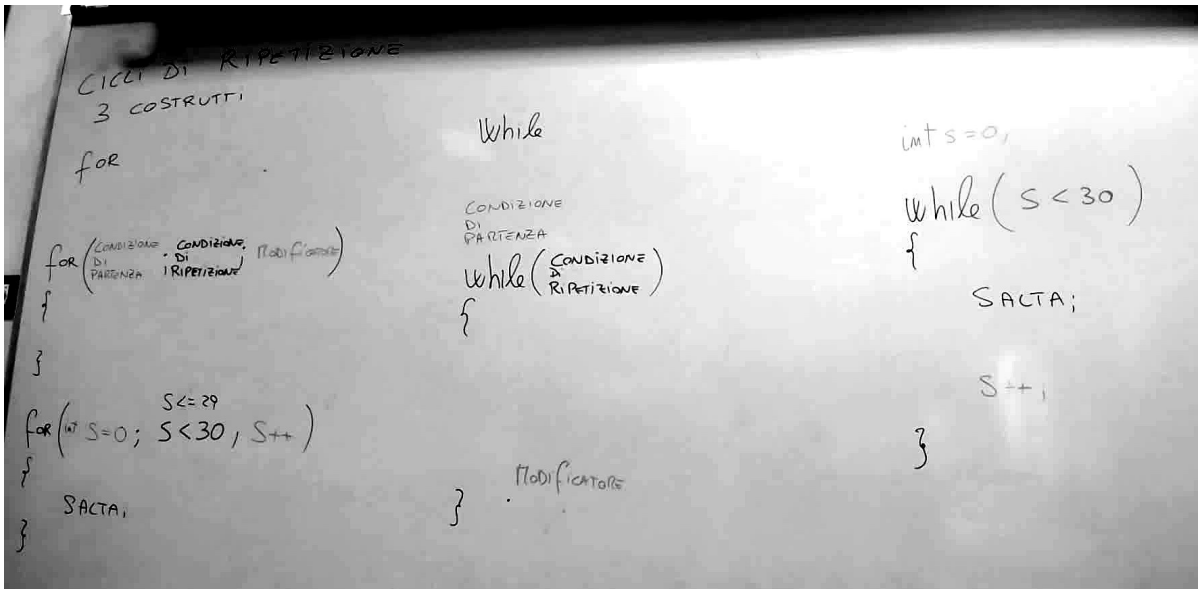
এই লুপ কতবার লাফায়?

```
for (s = 7; s < 30; s += 3)
{
    salta;
}
```

অনেকে শুধু 30 দেখে উত্তর দেয় "30 বার"। **এটি ঠিক নয়**: গণনা **7 থেকে শুরু হয়** এবং **3 করে** এগোয়।

s	7	10	13	16	19	22	25	28	31
s < 30?	হ্যাঁ	হ্যাঁ	হ্যাঁ	হ্যাঁ	হ্যাঁ	হ্যাঁ	হ্যাঁ	হ্যাঁ	**না**

লাফায় **8 বার**। এটি একটি চরম উদাহরণ, বাস্তবে প্রায় সবসময় 0 থেকে শুরু হয়, তবে এটি দেখায় যে পুনরাবৃত্তির সংখ্যা ঠিক করে **তিনটি অংশ** একসাথে।



ছবি/নকশা 5: বোর্ড: বাঁয়ে `for` (সাধারণ রূপ ও 30 লাফের উদাহরণ, বিকল্প হিসেবে `s <= 29`), মাঝে `while`-এর সাধারণ রূপ, ডানে উদাহরণ। একই রং মিলে যাওয়া অংশ বোঝায়।

> **সতর্কতা**
>
> শর্তটি **পুনরাবৃত্তি চালিয়ে যাওয়ার** শর্ত, থামার **নয়**।
>
> - `s < 30` আর `s <= 29` **সমতুল্য** (দ্বিতীয়টিতে শুধু একটি অক্ষর বেশি, আর প্রোগ্রামার অলস)।
> - `s == 29` **ভুল**: শুধু s 29 হলে পুনরাবৃত্তি করত, তাই 0 থেকে শুরু করলে **কখনো নয়**।

7.3 while

`while` for-এর **একই কাজ** করে, অন্যভাবে লেখা: শুরুর শর্ত লুপের **আগে**, পুনরাবৃত্তির শর্ত **বন্ধনীতে**, পরিবর্তক লুপের **ভেতরে**।

```
int s = 0;
while (s < 30)
{
    salta;
    s++;
}
```

"While" মানে **"যখন"**, "যতক্ষণ": s 30-এর ছোট **যতক্ষণ**, পুনরাবৃত্তি করো। প্রতি ঘূর্ণনে যন্ত্র ওপরে ফিরে আবার **জিজ্ঞেস করে** `s < 30` কিনা; s 30 হলে **বেরিয়ে যায়**। পরিবর্তক `s++` সাধারণত **শেষে** থাকে, তবে লুপ কেমন আচরণ করবে তা দেখে সরানো যায়। ফ্লোচার্ট ছবি 4-এর মতো।

7.4 for না while? শৈলীর ব্যাপার

যেটি পছন্দ ব্যবহার করা যায়: `for` দিয়ে যা করা যায় `while` দিয়েও করা যায়, উল্টোটাও। তবে একটি **শৈলীর প্রবণতা** আছে:

সংজ্ঞা

- `for`: যখন কতবার পুনরাবৃত্তি হবে তা **আগে থেকে** জানা, অর্থাৎ **প্রোগ্রামার ঠিক করে** (10টি লাফ দাও: সবসময় 10)।
- `while`: যখন কতবার তা **জানা নেই**, কারণ প্রোগ্রাম চলার সময় **যা ঘটে** তার ওপর নির্ভর করে, হয়তো ব্যবস্থার **বাইরে** (ক্লান্ত না হওয়া পর্যন্ত লাফাও: কত লাফ হবে জানা নেই)।

উদাহরণ

- ভুল জানাতে একটি বাতি **10 বার জ্বালানো-নেভানো**: শৈলী অনুযায়ী `for`।
- ব্যবহারকারী **বোতাম না চাপা পর্যন্ত** বাতি জ্বালানো-নেভানো: `while`, কারণ কতবার তা জানা নেই।

উদাহরণ

দুটি লুপ মেলানো। "30টি বুকডনের সেট দাও, সেটের মাঝে বিরতি, আর ভেঙে না পড়া পর্যন্ত চালিয়ে যাও": বাইরে একটি `while` ("ভেঙে না পড়া পর্যন্ত চলো") আর ভেতরে একটি `for` ("30টি বুকডন দাও"), সাথে বিরতি। দুটি `for`, দুটি `while`, একটি `for` ও একটি `while` মেলানো যায়: **যা দরকার**, তিনটি অংশ ও তাদের ঠিক করা বোঝার পর।

টীকা

শিক্ষক 25 বছর শুধু `for` ব্যবহার করে প্রোগ্রাম করেছেন; পরে দরকারে `while` ব্যবহার করতে শিখেছেন। দুটোই জানা জরুরি, কারণ একজন প্রোগ্রামারকে **অন্যদের কোডও** পড়তে হয়: দলে সবসময় কেউ একটি, কেউ অন্যটি পছন্দ করে।

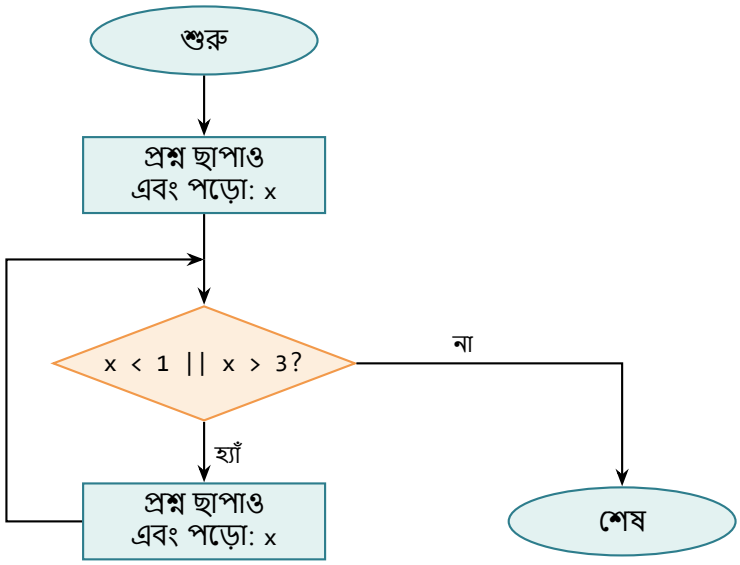
7.5 do while

সমস্যা। একটি প্রোগ্রাম লিখি যা ব্যবহারকারীর কাছে **1 থেকে 3 একটি সংখ্যা** চায়। ব্যবহারকারী অন্য মান দিলে (7, বা -5) প্রোগ্রাম **মেনে নেয় না** এবং **প্রশ্ন আবার করে**, সংখ্যা ঠিক না হওয়া পর্যন্ত। কতবার লাগবে জানা নেই: তাই একটি `while`। (একটি `if` বা `switch` সংখ্যা ঠিক কিনা দেখতে পারত, কিন্তু প্রশ্ন করত **একবারই**।)

```c
int x;
printf("Inserisci un numero da 1 a 3: ");
scanf("%d", &x);

while (x < 1 || x > 3)
{
    printf("Inserisci un numero da 1 a 3: ");
    scanf("%d", &x);
}
```

এখানে **পরিবর্তক** `s++` নয়: এটি ব্যবহারকারীর **নতুন উত্তর**, যা `x` বদলায়। লুপ চলে **যতক্ষণ** `x` 1-এর ছোট **অথবা** 3-এর বড়। লুপের পরে একটি বার্তা যোগ করা যায়, যেমন "numero corretto" (সংখ্যা ঠিক): এটি প্রোগ্রামারের পছন্দ।



ছবি/নকশা 6: `while`-এ প্রশ্ন দুবার থাকে: একবার লুপের আগে, একবার ভেতরে।

সতর্কতা

এই সংস্করণের সমস্যা: `printf` ও `scanf` **দুবার** **লেখা**। `while` লুপে ঢোকার **আগে** শর্ত যাচাই করে, তাই `x`-এর আগে থেকেই মান থাকতে হবে: **লুপের** **বাইরে** একটি প্রথম প্রশ্ন লাগে। এটি সরালে এবং `x`-এ ঘটনাক্রমে বৈধ সংখ্যা (যেমন 2) থাকলে লুপ **বাদ** **পড়ত** আর প্রশ্ন **কখনো** **হতো** **না**। একই জিনিস দুবার লেখা খারাপ: `do while` ঠিক এই সমস্যাটি মেটায়।

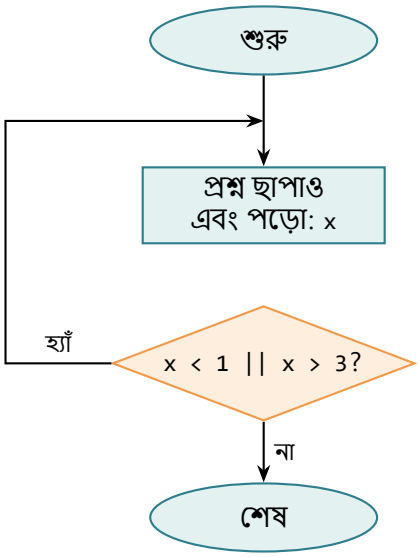
```c
int x;
do
{
    printf("Inserisci un numero da 1 a 3: ");
    scanf("%d", &x);
} while (x < 1 || x > 3);
```

সংজ্ঞা

`do while`-এ ব্লক চালানোর পরে **শেষে** শর্ত যাচাই হয়। তাই লুপের ভেতরের কোড **অন্তত** **একবার** চলে, শর্ত শুরু থেকেই মিথ্যা হলেও। `do while` `while`-এর বন্ধনীর পরে **সেমিকোলন** দিয়ে শেষ হয়।

উদাহরণ

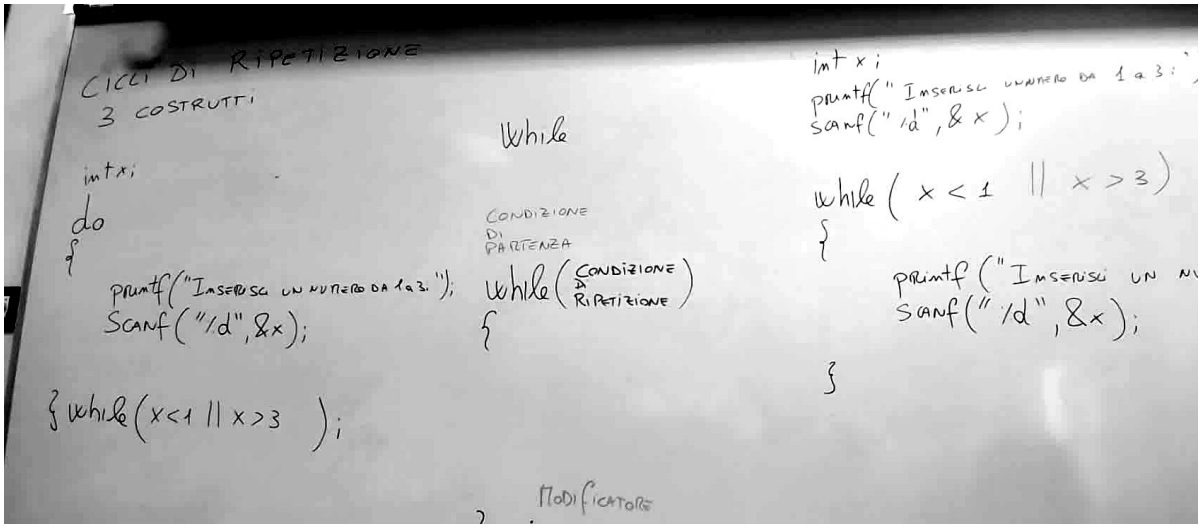
টাকার তুলনা। **`while`**-এ কিছু কিনতে যাওয়ার আগে তোমাকে জিজ্ঞেস করি কত টাকা দেবে, আর **আগে** **দেখি** যথেষ্ট কিনা: হলে যাই। **`do while`**-এ **সঙ্গে** **সঙ্গে** তোমাকে 5 ইউরো দিই, আর **পরে** দেখি যা কিনবে তার জন্য যথেষ্ট কিনা: না হলে আবার শুরু।



ছবি/নকশা 7: `do while`-এ প্রশ্ন একবারই লেখা: আগে চালানো, পরে যাচাই।

সতর্কতা

পুনরাবৃত্তির শর্ত ও "সংখ্যা ঠিক" শর্ত। `x < 1 || x > 3` মানে "**সংখ্যা ভুল**": এটি প্রশ্ন **আবার করার** শর্ত। "**সংখ্যা ঠিক**" শর্ত হলো `x >= 1 && x <= 3`, যা এর **উল্টো**, সমতুল্য রূপ নয়: **De Morgan**-এর উপপাদ্যে `x < 1 || x > 3` সমান `!(x >= 1 && x <= 3)`। লুপে লিখতে হয় **চালিয়ে যাওয়ার** শর্ত: তাই `x < 1 || x > 3`, অথবা `!(x >= 1 && x <= 3)`। ক্লাসে "`x >= 1 && x <= 3` সমতুল্য" লেখা থাকলে এভাবে ঠিক করে নাও।



ছবি/নকশা 8: বোর্ড: বাঁয়ে `do while`, ডানে `while` সংস্করণ, যেখানে `printf` ও `scanf` লুপের আগে ও ভেতরে পুনরাবৃত্ত।

সংজ্ঞা

সারসংক্ষেপ: `for`, `while` ও `do while` তিনটিই **পুনরাবৃত্তি করে**। `for` ও `while` প্রায় **অদলবদলযোগ্য**: **আগে** ঠিক করে পুনরাবৃত্তি হবে কিনা। `do while` ঠিক করে **পরে**: কোড **অন্তত একবার** চালায়। অন্য দুটির চেয়ে কম ব্যবহার হয়, তবে এটি আছে জানা দরকার।

8 onlinegdb-তে পরীক্ষা করা উদাহরণ

`printf("*");` একটি **তারকাচিহ্ন** ছাপায়। পুনরাবৃত্তিতে তারকাচিহ্ন **পাশাপাশি** ছাপে; `\n` দিলে **নতুন লাইনে**, একটির নিচে একটি। ক্লাসে নির্দেশটি কয়েকবার লিখে শুরু হয়েছিল:

```
printf("*\n");
printf("*\n");
printf("*\n");
printf("*\n");
printf("*\n");
printf("*\n");
printf("*\n");
printf("*\n");
```

কয়টি তারকাচিহ্ন? **8**টি, একটির নিচে একটি। আরও বুদ্ধিমানভাবে লেখা মানে একটি **লুপ** ব্যবহার করা:

```
#include <stdio.h>

int main()
{
    for (int s = 0; s < 180; s++)
    {
        printf("*");
    }
    return 0;
}
```

180টি তারকাচিহ্ন ছাপায় (ফ্লোচার্ট ছবি 4-এর মতো, 30-এর বদলে 180 আর `salta`-র বদলে `printf("*")`)। তারপর একই গণনা `while` দিয়ে, 30 পুনরাবৃত্তিতে (`s` 0 থেকে 29), এবং সমতুল্য `for` দিয়ে লেখা হয়েছে: ফলাফল **একই**, 30টি তারকাচিহ্ন। এরপর থেকে `printf`-এ `\n` যোগ হয়েছে, নতুন লাইনে গিয়ে ভালোভাবে দেখতে। ক্লাসে কিছু রূপ পরীক্ষা হয়েছে (এখানে সব `for` দিয়ে লেখা):

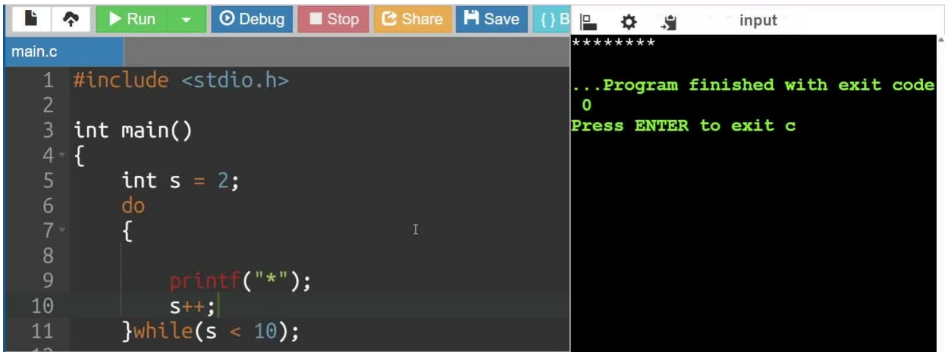
লুপ	`s`-এর মান	তারকাচিহ্ন
`for (s = 0; s < 15; s++)`	0, 1, 2, …, 14	15
`for (s = 0; s <= 15; s++)`	0, 1, 2, …, 15	16
`for (s = 0; s < 15; s += 2)`	0, 2, 4, 6, 8, 10, 12, 14 (2-এর নামতা)	8
`for (s = 2; s < 15; s += 3)`	2, 5, 8, 11, 14	5
`for (s = 0; s < 15; s += 15)`	0	1

8.1 অসীম লুপ

তারপর `do while`, প্রথমে **পরিবর্তক ছাড়া**:

```
int s = 2;
do
{
    printf("*");
} while (s < 10);
```

কতবার ছাপে? **অসীমবার**: কোথাও লেখা নেই `s` **কীভাবে বদলাবে**, তাই `s` 2-ই থাকে এবং শর্ত **সবসময় সত্য**। দশ বছর পরে ফিরলেও প্রোগ্রাম চলতে থাকবে। পরিবর্তক ছিল না: লুপের ভেতরে `s++;` যোগ করলে `s` 2 থেকে 9 পর্যন্ত যায় এবং তারকাচিহ্ন হয় **8**টি (ছবি 9)। ফ্লোচার্ট আগের অংশের `do while`-এর মতো: ব্লক চলে, তারপর `s < 10` যাচাই।



ছবি/নকশা 9: পরিবর্তক `s++` সহ `do while`: 2 থেকে শুরু করে `s < 10` যতক্ষণ, 8টি তারকাচিহ্ন ছাপে।

টীকা

কখনো অসীম লুপই চাই। বিমানের **অনবোর্ড** **কম্পিউটার** চালু থেকে বন্ধ হওয়া পর্যন্ত তার প্রোগ্রাম চালায়: প্রোগ্রাম শেষে পৌঁছে থেমে গেলে বিমান নিয়ন্ত্রণহীন হতো। **যেসব** **ব্যবস্থা** **কখনো** **থামবে** **না** সেখানে ইচ্ছে করে একটি **অসীম** `while` ব্যবহার হয়, যাতে ভেতরের সব চিরকাল চলে।

9 ক্লাসের প্রশ্ন

টীকা

- **সোর্স ও প্রোগ্রাম।** **সোর্স** হলো **কোড**, যা আমরা লিখি কিছু ঘটানোর জন্য। মানুষ ব্যবহার করে **চালানোর** **উপযোগী** **প্রোগ্রাম**। `.exe` ফরম্যাট **Windows**-এর প্রোগ্রামের: অন্য অপারেটিং সিস্টেম অন্যভাবে প্রোগ্রাম চালু করে। অনেক প্রোগ্রাম (যেমন স্কুলের লগইন পাতা) **বাণিজ্যিক** **পণ্য**: যে কেনে সে প্রোগ্রাম পায়, প্রজেক্ট open source না হলে **সোর্স** **পায়** **না**।
- **Linux-এও কি প্রোগ্রাম করা হয়?** হ্যাঁ: এই ভিত্তি **Linux**, **Windows** ও **Mac**-এ খাটে, এবং **যেকোনো** **ভাষায়** পাওয়া যায়: C, Python, Rust... রূপ একটু বদলায়, কিন্তু **ধারণা** একই।
- **শিক্ষক কোন ভাষা ব্যবহার করেন?** প্রধানত **C** ও **C#**।
- **লেখায় রং দেওয়া যায়?** প্রশ্নটি খুব বিস্তৃত: কোথায়, কীভাবে, কেন তার ওপর নির্ভর করে। Word নথির জন্য তৈরি ফাংশন আছে; আমাদের প্রোগ্রামের ছাপানো লেখার জন্য করা যায়, তবে সরঞ্জামের ওপর নির্ভর করে। নির্দিষ্ট উত্তর পেতে প্রশ্ন **নির্দিষ্ট** হতে হবে।
- **আর কৃত্রিম বুদ্ধিমত্তা?** দ্রুত কাজে অনেক সাহায্য করে, কিন্তু IA-র লেখা কোড **নিখুঁত** **নয়**: সবসময় **আবার** **পড়তে** ও হাত লাগাতে হয়।

10 অনুশীলনের জন্য

উদাহরণ

পরের পাঠে `for`, `while` ও `switch` নিয়ে **অনুশীলন** হবে। যে চায় বাড়িতে আগেই চেষ্টা করতে পারে, যেমন onlinegdb-তে: **কোনো জমা নেই**, আর না পারলে অসুবিধা নেই, ক্লাসে একসাথে করা হবে। (শিক্ষক কখনো এক দিনের জন্য কাজ দেন না।) নিচের অনুশীলনগুলো **এই পাঠ্যের প্রস্তাব**, ক্লাসে দেওয়া নয়:

1. একটি `for` দিয়ে এক লাইনে 20টি তারকাচিহ্ন ছাপাও, তারপর একটির নিচে একটি করে 20টি।
2. 5-এর নামতা (5, 10, 15, …, 50) একটি `for` দিয়ে ছাপাও, তারপর একটি `while` দিয়ে।
3. `switch` দিয়ে একটি মেনু: 1 = "Buongiorno", 2 = "Buonasera", 3 = "Buonanotte", নাহলে "Scelta non valida"।
4. একটি `do while` দিয়ে 1 থেকে 10 একটি সংখ্যা চাও, সংখ্যা বৈধ না হওয়া পর্যন্ত প্রশ্ন আবার করে।

11 এই পাঠ থেকে যা মনে রাখতে হবে

- `x = x + a` লেখা যায় **`x += a`**; একইভাবে `-=`, `*=`, `/=`।
- `x++` ও `x--` **1** যোগ বা বিয়োগ করে। **`++x`** মান ব্যবহারের **আগে** বাড়ায়, **`x++`** **পরে**: `x = 7` হলে `y = x++` দেয় `y = 7`, `y = ++x` দেয় `y = 8`; দুই ক্ষেত্রেই `x = 8`।
- **`switch`** একটি ভেরিয়েবলকে **`case`**-গুলোর সাথে তুলনা করে; **`break`** গঠন থামায়; `break` ছাড়া পরের ক্ষেত্রগুলোতে **চলতে থাকে**; **`default`** বাকি সব মান ধরে।
- **`for`**-এর তিন অংশ **সেমিকোলনে** আলাদা: শুরু, পুনরাবৃত্তির শর্ত, পরিবর্তক। `for (int s = 0; s < 30; s++)` **30 বার** চলে।
- শর্ত হলো **চালিয়ে যাওয়ার**, থামার নয়।
- পুনরাবৃত্তির সংখ্যা **আগে ঠিক করা** হলে **`for`**, **যা ঘটে তার ওপর নির্ভর করলে** **`while`**।
- **`do while`** **শেষে** শর্ত যাচাই করে: কোড **অন্তত একবার** চালায়; **`;`** দিয়ে শেষ হয়।
- পরিবর্তক ছাড়া লুপ **অসীম**: সাধারণত এটি ভুল, তবে যেসব ব্যবস্থা কখনো থামবে না সেখানে ইচ্ছে করে ব্যবহার হয়।

12 শব্দকোষ: ক্লাসে ব্যবহৃত ইতালীয় শব্দ

- **incremento / decremento**: বাড়ানো / কমানো
- **assegnazione composta**: যৌগিক মান নির্ধারণ
- **ciclo**: লুপ
- **scelta**: পছন্দ
- **Inserisci una scelta**: একটি পছন্দ দাও

- **Hai scelto la prima / seconda opzione**: তুমি প্রথম / দ্বিতীয় বিকল্প বেছেছ
- **Hai fatto la terza scelta**: তুমি তৃতীয় পছন্দ করেছ
- **Inserisci un numero da 1 a 3**: 1 থেকে 3 একটি সংখ্যা দাও
- **salto / salta**: লাফ / লাফ দাও
- **partenza / passo**: শুরু / ধাপ
- **tabellina**: নামতা
- **Buonanotte**: শুভ রাত্রি